

AI Influencer Generator & Auto Post Scheduler

A comprehensive production guide detailing subsystems, secure pipelines, database modeling, external API frameworks, and DevOps operations from starting to end.

AUTHOR / OWNER

Shruti Sania

CURRENT RELEASE TIER

Production Release v1.0.0

PRIMARY STACK

Next.js 16 | Supabase | Stripe | Zernio

PUBLISHED ON

May 18, 2026

1. Platform Overview & Design Philosophy

The **AI Influencer Generator & Auto Post Scheduler** is a state-of-the-art software-as-a-service (SaaS) application engineered to empower content creators, digital marketers, and agencies to construct custom synthetic personas (AI Influencers) and automate their presence across key global social networks.

Unlike traditional post scheduling queues, this platform integrates generative media directly into the scheduler. Content creators do not simply schedule text or pre-made pictures; they construct a multi-dimensional digital model that is represented in the database as a visual identity. The system maintains visual consistency across posts by compiling visual character parameters, generating appropriate custom campaign graphics, drafting optimized social media captions, and queuing them to real accounts.

1.1 Core Architecture Principles

- **Consistent Identity Rendering:** Generates portraits and full-body model variations using precise generative descriptions, bypassing identity drifting between individual renders.
- **Strict Resource Tokenomics:** Incorporates credit-based quotas (50 credits for character design, 20 credits for post generation) verifying balances prior to billing external APIs.
- **Robust Schema Agnosticism:** Implements deep-level try-catch models and sandboxed fallbacks so that database updates or missing API integrations never break the dashboard experience.
- **Intuitive Client Dashboards:** Implements sleek, responsive visual indicators, hover states, drag-and-drop calendar queues, and inline subscription simulations.

2. The Core Technical Stack

The platform features a modular, modern developer environment chosen for high performance, ease of deployment, and modern standards:

Subsystem	Technology Choice	Architectural Purpose
Client & Server Core	Next.js 16 (App Router) & React 19	Server-Side Rendering, Server Actions, and high-performance hydrates.
Database & Storage	Supabase (PostgreSQL & S3 Storage)	Session authorization, real-time relational tables, and hosted media storage.
AI Media Engine	Fal.ai (API client), Hugging Face, Luma Agents	Consistent-character image generation pipelines and video synthesis.
SaaS Monetization	Stripe API (Dynamic Price Passing)	Inline checkout pages, asynchronous webhooks, and sandbox payments.
Social Scheduling	Zernio API (Unified Multi-Platform Gateway)	OAuth account linkage, media presigning, and immediate/delayed posting.
Styling UI/UX	Tailwind CSS v4 & Glassmorphism CSS	Harmonious dark-mode UI with blur filters, responsive widgets, and grids.

 Developer Note: React 19 Hydration Compliance

React 19 strict typing requires components like calendars and sidebars to be highly resilient against mismatched client-server hydration states. The platform uses explicit React hooks and CSS loaders to keep pages loading flawlessly.

3. Comprehensive Database Schema (SQL)

The data system is managed using **Supabase (PostgreSQL)**. The structural links are governed by CASCADE constraints ensuring data integrity. If a user deletes their account, all related custom models, campaign posts, and social tokens are cleaned up securely.

3.1 Complete SQL Table Setup

Execute the following SQL instructions inside your Supabase SQL Editor to establish the relational schemas:

```
-- 1. USERS TABLE
CREATE TABLE public.users (
  id UUID PRIMARY KEY REFERENCES auth.users ON DELETE CASCADE,
  email TEXT,
  full_name TEXT,
  avatar_url TEXT,
  credits INTEGER DEFAULT 300, -- Entered on standard free tier regi
  subscription_tier TEXT DEFAULT 'free', -- 'free', 'standard', 'pro'
  subscription_status TEXT DEFAULT 'inactive', -- 'active', 'inactive', 'canceled'
  stripe_customer_id TEXT,
  stripe_subscription_id TEXT,
  updated_at TIMESTAMP WITH TIME ZONE
);

-- 2. MODELS TABLE (AI INFLUENCERS)
CREATE TABLE public.models (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES public.users(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  gender TEXT,
  body_type TEXT,
  skin_tone TEXT,
  age_range TEXT,
  hair_style TEXT,
  hair_color TEXT,
  eye_color TEXT,
  vibe TEXT,
  prompt TEXT,
  portrait_url TEXT, -- Signed secure storage link
  full_body_url TEXT, -- Signed secure storage link
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL
);

-- 3. POSTS TABLE (GENERATED MEDIA CAMPAIGNS)
```

```

CREATE TABLE public.posts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES public.users(id) ON DELETE CASCADE,
  model_id UUID REFERENCES public.models(id) ON DELETE CASCADE,
  prompt TEXT NOT NULL,          -- JSON payload containing schedules, subtitles, and
  image_url TEXT,                -- Output CDN asset link
  aspect_ratio TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL
);

-- 4. SOCIAL ACCOUNTS TABLE
CREATE TABLE public.social_accounts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES public.users(id) ON DELETE CASCADE,
  platform TEXT NOT NULL,        -- 'instagram', 'linkedin', 'tiktok', 'youtube', 'pin
  profile_id TEXT NOT NULL,      -- Sync key generated from Zernio
  account_id TEXT NOT NULL,      -- Unique platform channel identifier
  account_name TEXT,
  avatar_url TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL,
  CONSTRAINT unique_user_platform_account UNIQUE (user_id, platform, account_id)
);

```

3.2 Architectural Table Relationships

The architecture follows a strict 1-to-many cascading parent structure:

- Every `public.models` and `public.posts` belongs directly to an authenticated `public.users` primary key.
- Each post generated belongs to a target `public.models` (AI Influencer), establishing visual alignment.
- The `unique_user_platform_account` constraint prevents users from linking the exact same Instagram or LinkedIn channel multiple times, protecting token synchronization.

4. Backend API Pipelines & Server Actions

The application utilizes Next.js Server Actions to securely process calculations on the server, avoiding exposing sensitive API keys to the client.

4.1 AI Influencer Studio Generator

The generation file `app/actions/influencers.ts` orchestrates the avatar and full-body rendering pipeline:

1. **Credit Auditing:** Validates that the active user profile has at least **50 credits**. If not, it returns an early error response.
2. **Generative Synthesis:** Formulates highly structured textual prompts based on the UI choices and invokes the `@fal-ai/client`.
3. **Secure Storage Buffer:** Downloads the binary image buffer from Fal.ai CDN and transfers it to the Supabase Storage Bucket under `public/models/${userId}/${timestamp}.jpg`.
4. **Profile Refills & DB Insertion:** Deducts 50 credits, updates the user's table, and writes the character parameters to the `models` table.

4.2 Zernio Unified Social OAuth Integrator

The social scheduler file `app/actions/social.ts` interfaces with the Zernio Social API:

- **Profile Provisioning:** Uses `getOrCreateZernioProfile` to map users to independent Zernio nodes, securing profile integrity.
- **Fallback Mock Linker:** If a developer is working offline or hasn't configured a `ZERNIO_API_KEY`, the server actions fallback to a robust **Mock Linker Simulator**. This returns mock keys and logs active simulator badges, enabling the front-end features to run smoothly in sandboxed sandboxes.
- **Unified Post Publishing:** Pushes scheduled assets to Zernio's CDN presigned bucket (`/media/presign`), downloads the file streams, and executes publishing updates for scheduled times.

4.3 Stripe Subscription Gateway

The payment pipeline in `app/actions/stripe.ts` features elegant developer shortcuts:

```
// Dynamic Stripe Product Checkout Configuration
const session = await stripe.checkout.sessions.create({
  payment_method_types: ['card'],
  line_items: [{
    price_data: {
      currency: 'usd',
      product_data: {
        name: 'Pro Premium Monthly Plan',
        description: '3000 monthly credits & unlimited multi-platform posting',
      },
      unit_amount: 4900, // $49.00 USD
      recurring: { interval: 'month' }
    },
    quantity: 1,
  }],
  mode: 'subscription',
  success_url: `${origin}/dashboard/settings?session_id={CHECKOUT_SESSION_ID}`,
  cancel_url: `${origin}/dashboard/settings`,
});
```

This inline method removes the overhead of manually pre-creating pricing items in the Stripe Dashboard, making setup fast and fully dynamic.

5. Frontend UI/UX & Interactive Calendar

The platform features a visually stunning visual layout styled with premium glassmorphism, responsive navigation, and beautiful states.

5.1 Interactive Scheduling Calendar Dashboard

The core user dashboard features a drag-and-drop calendar scheduler that coordinates your publishing timeline:

- **Draft Selector Dialog:** Clicking on any empty day slot triggers a beautiful hover modal. Users can inspect existing unpublished campaign drafts, edit the accompanying post description or scheduled publishing hour, and save changes immediately.
- **Aesthetic Hover Thumbnails:** Moving the cursor over scheduled items instantly renders a micro-animation previewing the visual graphic and platform indicators.
- **Analytics Insight Panel:** The calendar footer contains mathematical metrics analyzing weekly/monthly scheduled items, distribution percentages (e.g., "Instagram: 40%, LinkedIn: 60%"), and warning notifications for days with heavy queues.

5.2 Billing Dashboard & Credit Simulator

The billing page (`app/dashboard/settings/page.tsx`) handles subscription configuration. If the developer doesn't have an active Stripe integration, the page provides a **Credit Card Checkout Simulator Modal**. Users can enter mock credentials, trigger a loading state, and safely credit their local profiles with subscription balances without sending real network requests.

6. Operations, Setup & Deployment Manual

Follow this complete operational guide to configure the application from scratch and deploy it to a live hosting environment.

6.1 Local Installation Sequence

Run these terminal instructions in your workspace:

```
# 1. Download project files
git clone https://github.com/shrutisania11/ai-influencer-generator.git
cd ai-influencer-generator

# 2. Setup standard packages
npm install

# 3. Spin up hot-reloading development server
npm run dev
```

6.2 System Environment Variables

Create a file in your project root named `.env.local` and populate the credentials:

```
# -----
# DATABASE CONFIGURATION (Supabase Dashboard Settings)
# -----
NEXT_PUBLIC_SUPABASE_URL=https://your-project-id.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

# -----
# BILLING CONFIGURATION (Stripe Developer Dashboard)
# -----
STRIPE_SECRET_KEY=sk_test_51...
STRIPE_WEBHOOK_SECRET=whsec_...

# -----
# MEDIA ENGINES CONFIGURATION (Fal.ai Dashboard)
# -----
FAL_KEY=fal-key-string...

# -----
```

```
# SOCIAL CHANNEL GATEWAY CONFIGURATION (Zernio Console)
# -----
ZERNIO_API_KEY=zc_key_...
```

⚠ Security Protocol: API Key Exposure

Never commit `.env.local` to GitHub. If you accidentally upload your credentials, IMMEDIATELY navigate to the respective developer console, delete the compromised secret, and request a replacement token.

6.3 Development Webhook Tunneling

Because Stripe utilizes asynchronous server callbacks to tell your database that a subscription payment was successful, you must run a webhook listener in development:

1. Download and install the **Stripe CLI** on your computer.
2. Log in to your account: `stripe login`
3. Start the event forwarding pipeline:

```
stripe listen --forward-to localhost:3000/api/webhooks/stripe
```

4. Copy the returned webhook signing secret (starts with `whsec_`) and paste it as `STRIPE_WEBHOOK_SECRET` in your local `.env.local` file.

7. Git CLI & GitHub Profile Optimization Guide

Once the code is ready, presenting it in an organized, beautiful manner on GitHub is essential for proving your full-stack engineering proficiency.

7.1 Complete Git Sync Procedures

If your local code gets out of sync with your remote GitHub repository (for example, if you edit or delete files like `CLAUDE.md` on the web interface), you will receive a *"push rejected"* warning. Follow this precise sequence to synchronize and publish your changes safely:

```
# 1. Stage your local changes
git add .

# 2. Create a clean commit message
git commit -m "docs: polish README and add author credit"

# 3. Pull and integrate remote web updates (keeps history aligned)
git pull origin main --rebase

# 4. Push local changes back to GitHub securely
git push origin main
```

7.2 Repository Optimization Protocol

- **Interactive Description:** Populate the repository's "About" section with an active description and link to your live Vercel landing page.
- **Targeted Topic Tags:** Add discoverability tags including `nextjs`, `supabase`, `stripe`, `generative-ai`, and `saas`.
- **Profile Pinning:** Pin the repository at the top of your GitHub homepage, so it is the primary item recruiters see.